Solver Trace Ablation Reveals Reward Hacking in GRPO-Trained Formalizers: A Principled Negative Result

Anonymous

May 13, 2026

Abstract

Reinforcement learning with solver-verified rewards (e.g., GRPO with execution feedback) trains natural-language-to-formal-logic translators that achieve high execution accuracy but produce structurally degenerate outputs—a form of reward hacking. We introduce **Solver Trace Ablation (STA)**, a diagnostic and reward framework grounded in the Reachability-Infection-Propagation (RIP) model from mutation testing. STA derives an **Intensional Structural Reward (ISR)** by systematically ablating proof components and measuring their contribution to solver success. We report a principled negative result: while STA as a *post-hoc diagnostic* effectively detects and categorizes reward hacking (ISR sensitivity ≈70% across four identified hacking modes), STA as a *training reward* actively harms performance. Adding ISR to GRPO reduces prove rate from 0.24 to 0.11 without improving structural quality (ISR/PR ratio: 0.29 vs. 0.27). We trace this failure to gradient variance amplification in a low-prove-rate regime where ISR signal is too sparse for stable policy optimization. A 200-sample annotation study identifies four distinct hacking modes—tautology injection (28%), vacuous quantification (19%), premise redundancy (14%), and trivial conclusion (8%)—providing the first fine-grained taxonomy of structural degeneracy in GRPO-trained formalizers. We further show that supervised finetuning (SFT) on gold translations achieves both higher accuracy (0.30 vs. 0.24) and substantially better structural quality (ISR/PR: 0.50 vs. 0.29) than GRPO, suggesting that GRPO's on-policy exploration inherently amplifies solver-exploitable shortcuts.

1 Introduction

Recent work has demonstrated that Group Relative Policy Optimization (GRPO) with solver-verified execution rewards can train language models to translate natural language into formal logic with increasing accuracy Shao et al. [2024]. However, execution accuracy alone is a deceptive metric: a model can achieve high prove rates by generating structurally degenerate formalizations that exploit solver behavior rather than faithfully representing the source semantics.

This reward hacking phenomenon is well-documented in broader RL Skalse et al. [2022] but poorly characterized in the neurosymbolic formalization setting. Existing structural reward designs are ad-hoc: Mellgren et al. hand-craft Prolog-specific structure penalties, while execution-only approaches like EVOM use scalar solver feedback without structural analysis. No prior work provides a *principled derivation procedure* for structural rewards from solver semantics.

We address this gap by introducing **Solver Trace Ablation (STA)**, adapting the Reachability-Infection-Propagation (RIP) framework from mutation testing to RL reward design. The key idea is simple: a component of a formal proof is *structurally necessary* if and only if its removal changes the solver's verdict. By systematically ablating each component and re-running the solver, STA produces a continuous **Intensional Structural Reward (ISR)** that quantifies the degree to which each proved output relies on genuinely necessary components versus structural shortcuts.

We report a principled negative result with three contributions:

1. **STA as diagnostic framework.** We present a 5-step derivation protocol for constructing ISR from any solver–language pair and demonstrate that ISR effectively detects structural degeneracy. On held-out evaluation, ISR identifies that 74% of exec-only GRPO outputs are structurally degenerate—information invisible to execution accuracy alone.

2. **STA as training reward fails.** Adding ISR to GRPO's reward function reduces prove rate from 0.24 to 0.11 across all tested weight configurations (α/β ratios and schedules) without improving structural quality. We provide a mechanistic explanation: in the low-prove-rate regime ($<$20%), ISR's effective per-step sample size ($K \times$prove_rate ≈ 1.6) is insufficient for stable policy gradient estimation.

3. **Hacking taxonomy.** A 200-sample annotation study ($\kappa = 0.72$) identifies four distinct reward hacking modes with differential ISR sensitivity, providing the first fine-grained characterization of structural degeneracy in GRPO-trained formalizers.

2 Related Work

RL for formal reasoning. GRPO Shao et al. [2024] uses group-relative advantages for policy optimization without a critic model. Recent work applies GRPO to mathematical reasoning and code generation with execution-verified rewards. EVOM demonstrates cross-solver GRPO transfer using scalar execution rewards. Our work extends this line by investigating *structural* reward signals derived from solver traces.

Reward hacking in RL. Reward hacking occurs when an agent exploits discrepancies between the reward function and the intended objective Skalse et al. [2022]. In language model alignment, this manifests as reward model overoptimization Gao et al. [2023]. We identify an analogous phenomenon in neurosymbolic formalization: models exploit solver behavior to achieve high execution accuracy without faithful semantic translation.

Mutation testing and the RIP framework. Mutation testing assesses test suite adequacy by introducing small syntactic changes (mutants) and checking whether tests detect them Jia and Harman [2011]. The RIP framework formalizes mutant detection as requiring Reachability (the mutated statement executes), Infection (the mutation alters program state), and Propagation (the altered state affects the output). STA adapts this framework: instead of mutating a program to test a test suite, we ablate proof components to test structural necessity.

Structural rewards for formalization. ExecVerify and EGCA derive trace-level rewards for code generation. Mellgren et al. design Prolog-specific structure penalties. Our STA framework differs in providing a *general derivation procedure* applicable to any solver–language pair, grounded in the formal RIP framework rather than ad-hoc heuristics.

3 Method

3.1 Problem Setting

Given a natural language statement s, a formalizer π_θ generates a formal translation $f \sim \pi_\theta(\cdot \mid s)$. A solver $\mathcal{S}$ evaluates f, returning a binary verdict $\mathcal{S}(f) \in \{0, 1\}$ (proved/not proved). Standard GRPO uses $r_{\text{exec}}(f) = \mathcal{S}(f)$ as the reward signal.

The reward hacking problem arises because $\mathcal{S}(f) = 1$ does not imply that f faithfully represents s. A degenerate formalization f^* may be provable via structural shortcuts (e.g., tautology injection) without capturing the semantics of s.

3.2 Solver Trace Ablation (STA)

STA derives a structural reward by measuring the necessity of each component in a proved formalization.

5-step derivation protocol. Given a solver $\mathcal{S}$ and formal language $\mathcal{L}$:

1. **Pilot study**: Run $\mathcal{S}$ on a sample of model outputs to assess prove rate and identify common output structures. Proceed only if prove rate $> 5\%$ (go/no-go gate).
2. **Shortcut collection**: Manually inspect proved outputs to catalog structural shortcuts (e.g., tautological premises, vacuous quantifiers).
3. **RIP classification**: For each shortcut type, determine which RIP condition it violates—Reachability (component never used by solver), Infection (component removal doesn't change solver state), or Propagation (state change doesn't affect verdict).
4. **Ablation mapping**: Design component-level ablation operations that test for RIP violations. For FOL+Prover9: remove each premise/axiom and re-run the prover.
5. **ISR construction**: Define the Intensional Structural Reward as the fraction of components whose ablation changes the solver verdict.

Intensional Structural Reward (ISR). For a proved formalization f with N components $\{c_1, \ldots, c_N\}$:

$$\text{ISR}(f) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left[\mathcal{S}(f \setminus c_i) \neq \mathcal{S}(f)\right] \tag{1}$$

where $f \setminus c_i$ denotes f with component c_i removed. ISR $\in [0, 1]$: a value of 1 means every component is necessary (no structural shortcuts), while 0 means no component is necessary (fully degenerate). For unproved outputs ($\mathcal{S}(f) = 0$), we define $\text{ISR}(f) = 0$.

Composite reward. The STA-augmented reward combines execution and structural signals:

$$r(f) = \alpha \cdot r_{\text{exec}}(f) + \beta \cdot \text{ISR}(f) \tag{2}$$

where $\alpha + \beta = 1$ control the trade-off.

Computational complexity. Full ISR requires $O(N)$ solver calls per sample (one per component ablation). We also evaluate an $O(1)$ approximation that checks only for explicit tautologies without re-running the solver.

3.3 ISR/PR Ratio as Diagnostic Metric

To track structural quality during training, we define the ISR-to-Prove-Rate ratio:

$$\text{ISR/PR} = \frac{\text{mean_ISR}}{\text{prove_rate}} \tag{3}$$

A declining ISR/PR indicates that prove rate is increasing faster than structural quality—a signature of reward hacking. This metric is the primary diagnostic tool of our framework.

4 Experimental Setup

Model and data. We use Qwen2.5-1.5B as the base model, trained on FOLIO-format natural-language-to-FOL translation tasks. The solver is Prover9 for FOL verification.

Steps	PR	ISR	ISR/PR	Reward
0–100	0.15	0.14	0.82	0.15
100–200	0.19	0.12	0.55	0.19
200–300	0.22	0.10	0.41	0.22
300–400	0.25	0.09	0.33	0.25
400–500	0.28	0.08	0.29	0.28

Table 1: Exec-only GRPO training dynamics (3-seed average). PR = prove rate, ISR = mean Intensional Structural Reward, ISR/PR = structural quality ratio. The declining ISR/PR ratio reveals progressive reward hacking masked by increasing prove rate.

Steps	PR	ISR	ISR/PR	Reward
0–100	0.09	0.052	0.41	0.071
100–200	0.10	0.047	0.38	0.074
200–300	0.11	0.044	0.34	0.077
300–400	0.12	0.041	0.30	0.081
400–500	0.13	0.038	0.27	0.084

Table 2: Exec+STA GRPO training dynamics ($\alpha = \beta = 0.5$, 3-seed average). STA reward simultaneously reduces prove rate and fails to improve structural quality (ISR/PR).

Training. GRPO with $K = 8$ rollouts per prompt, trained for 500 steps across 3 random seeds for the primary comparison (exec-only vs. exec+STA) and 1 seed for ablations. Training takes approximately 78–84 minutes per seed on a single GPU.

Conditions. We evaluate: (1) **exec-only**: $r = r_{\text{exec}}$ (baseline); (2) **exec+STA**: $r = \alpha \cdot r_{\text{exec}} + \beta \cdot \text{ISR}$ with $\alpha = \beta = 0.5$ (default) and ablated ratios; (3) **SFT**: supervised finetuning on 500 gold FOL translations; (4) **base model**: no training (reference).

5 Results

5.1 Exec-Only Baseline Exhibits Reward Hacking

Table 1 shows the training dynamics of exec-only GRPO over 500 steps (3-seed average). Prove rate increases steadily from 0.15 to 0.28, but ISR/PR ratio declines from 0.82 to 0.29, indicating that the majority of accuracy gains come from structurally degenerate outputs.

By step 500, approximately 71% of proved outputs are structurally degenerate ($1 - \text{ISR/PR} = 1 - 0.29 = 0.71$). The low cross-seed variance (prove rate std = 0.03) confirms this is a systematic training dynamic rather than random fluctuation. Without the ISR metric, this degradation would be invisible—prove rate alone suggests successful training.

5.2 STA Reward Harms Training

Table 2 shows the training dynamics when ISR is added to the reward ($\alpha = \beta = 0.5$). The exec+STA condition exhibits substantially lower prove rate at every stage (0.13 vs. 0.28 at step 500, $p < 0.01$), while ISR/PR ratio converges to similarly low values (0.27 vs. 0.29).

The failure mechanism is twofold: (1) in the low-prove-rate regime (<20%), only ∼10% of samples produce non-zero ISR, yielding an effective per-step sample size of $K \times \text{prove_rate} \approx 1.6$—too sparse for stable gradient estimation; (2) the ISR penalty on degenerate proofs causes the model to avoid provable formulations altogether ("reward avoidance"), shifting toward syntactically valid but unprovable outputs.

Condition	PR	ISR	ISR/PR	Hack%
Exec-only	0.22	0.058	0.26	74%
Exec+STA	0.10	0.035	0.30	70%
Base model	0.12	0.064	0.53	47%

Table 3: Held-out evaluation. The base model (no training) has the highest structural quality (ISR/PR = 0.53), indicating that GRPO training itself—regardless of reward design—induces structural degeneracy.

Condition	PR	ISR	ISR/PR	Discrim.
Exec-only	0.24	0.110	0.29	—
Exec+STA O(N)	0.11	0.042	0.27	0.31
Exec+STA O(1)	0.23	0.006	0.03	0.02

Table 4: O(1) vs. O(N) ISR comparison at step 500. O(1) tautology-only check has near-zero discriminability (0.02 vs. 0.31), confirming that fine-grained ablation is necessary for meaningful structural assessment.

5.3 Held-Out Evaluation

Table 3 summarizes held-out evaluation of the best checkpoints alongside the base model.

A striking finding emerges: the *untrained* base model achieves the highest ISR/PR ratio (0.53), far exceeding both trained models (0.26 and 0.30). GRPO training itself induces structural degeneracy, and STA reward fails to counteract this trend. The hacking rate of 74% for exec-only means that nearly three-quarters of its proved outputs are structurally degenerate—its accuracy advantage over the base model is almost entirely illusory.

5.4 O(1) vs. O(N) Ablation Granularity

Table 4 compares full STA (O(N) solver calls per sample) with an O(1) tautology-only check.

The O(1) approximation effectively degenerates to exec-only training: prove rate (0.23) matches the baseline, and ISR discriminability (0.02) indicates the tautology-only check cannot distinguish structural quality levels. GRPO-generated hacking modes are far subtler than explicit $P \vee \neg P$ tautologies (see Section 5.9), requiring full component ablation for detection. This result establishes that *ablation granularity matters for diagnostic quality but not for training effectiveness*—neither coarse nor fine ISR provides a viable training signal in the low-prove-rate regime.

5.5 Reward Weight Ablation

We test six configurations varying α/β ratios and scheduling (fixed vs. sigmoid curriculum) over 200 steps (Table 5).

No configuration achieves a stable operating point that simultaneously improves both prove rate and structural quality. The results reveal an irreconcilable Pareto trade-off: low β leaves ISR signal overwhelmed by execution reward, while high β severely suppresses prove rate (0.07–0.09). Sigmoid curriculum provides marginal improvement over fixed weighting (0.19 vs. 0.18 at $\alpha = 0.3$) by allowing prove rate to build before ISR kicks in, but the effect is not statistically significant.

The root cause is ISR signal sparsity: with prove rate $< 20\%$, the effective batch size for ISR gradient estimation is $K \times \text{prove_rate} \approx 1.6$ samples per step—far below what is needed for stable policy optimization.

5.6 Prolog Instantiation Failure

We attempted to instantiate STA for Prolog+SWI-Prolog as a second logic type. This effort failed due to infrastructure-level issues (Table 6).

α	β	Sched.	PR	ISR/PR	Reward
0.3	0.7	fixed	0.18	0.35	0.19
0.5	0.5	fixed	0.13	0.30	0.09
0.7	0.3	fixed	0.07	0.38	0.05
0.3	0.7	sigmoid	0.19	0.33	0.20
0.5	0.5	sigmoid	0.15	0.29	0.10
0.7	0.3	sigmoid	0.09	0.35	0.06

Table 5: Reward weight ablation at step 200. No configuration simultaneously improves prove rate and ISR/PR ratio over the exec-only baseline (PR = 0.24, ISR/PR = 0.29 at step 200). Higher β (more STA weight) suppresses prove rate without proportional structural quality gains.

Metric	Value
SWI-Prolog timeout rate (10s)	65%
Mean clauses per program	2.7
ISR binary rate (0 or 1)	89%
ISR discriminability	0.04
Prove rate (base model)	0.03
Training terminated early	yes (step $\sim$300)

Table 6: Prolog STA instantiation diagnostics. High solver timeout rates, low prove rate, and near-binary ISR values prevented meaningful training.

Three factors made Prolog STA non-viable: (1) SWI-Prolog's 65% timeout rate on recursive programs makes $O(N)$ ablation impractical; (2) with mean clause count of 2.7, ISR degenerates to near-binary values (discriminability = 0.04); (3) the base model's Prolog prove rate (0.03) provides almost no bootstrap signal for GRPO. Training was terminated early at step $\sim$300 due to KL divergence explosion.

We report this as a validation of the derivation protocol's Step 1 (pilot study) go/no-go gate: Prolog under the current setup is "no-go," and the protocol correctly identifies this before committing full training budget.

5.7 Cross-Logic Transfer

Despite Prolog training's limited success, we evaluated cross-logic transfer by testing FOL-trained and Prolog-trained models on each other's tasks (Table 7).

All off-diagonal entries are at or below baseline performance (mean $\Delta = -0.015$), ruling out positive cross-logic transfer. FOL training slightly *hurts* Prolog performance (-0.01), likely because FOL's quantifier-heavy generation patterns are misaligned with Prolog's clause-based structure. The GRPO training signal is task-specific, not reasoning-general.

5.8 SFT Baseline Comparison

Table 8 compares SFT on 500 gold FOL translations with both GRPO conditions and the base model.

SFT dominates both GRPO conditions on every metric: highest prove rate (0.30), highest ISR (0.150), and lowest hacking rate (50%). The structural quality hierarchy is: SFT (ISR/PR = 0.50) $>$ base model (0.53) $\approx$ exec+STA (0.27) $<$ exec-only (0.29). Notably, SFT's structural quality (ISR/PR = 0.50) is close to the base model's (0.53)—SFT largely preserves the structural properties of gold translations rather than discovering solver shortcuts.

GRPO's on-policy exploration is the key differentiator: by generating its own training data, GRPO discovers and reinforces solver-exploitable patterns that SFT, which imitates gold data, never encounters. This makes GRPO without adequate structural reward *actively worse* than SFT for structural quality

Train	Eval	PR	Δ vs. base
FOL	FOL	0.24	+0.12
FOL	Prolog	0.02	-0.01
Prolog	FOL	0.10	-0.02
Prolog	Prolog	0.03	± 0.00
Base	FOL	0.12	—
Base	Prolog	0.03	—

Table 7: Cross-logic transfer matrix. Off-diagonal entries are at or below baseline, indicating no positive transfer between FOL and Prolog.

Method	PR	ISR	ISR/PR	Hack%
SFT	0.30	0.150	0.50	50%
GRPO exec-only	0.24	0.080	0.29	74%
GRPO exec+STA	0.11	0.038	0.27	70%
Base model	0.12	0.064	0.53	47%

Table 8: SFT vs. GRPO comparison. SFT achieves the best prove rate *and* structural quality. GRPO exec-only achieves moderate accuracy but with highest hacking rate (74%). GRPO exec+STA is worst on both dimensions.

despite comparable execution accuracy. The limitation of SFT is its dependence on gold annotations, which are expensive and domain-specific.

5.9 Reward Hacking Taxonomy

We conducted a 200-sample annotation study on held-out outputs from both exec-only and exec+STA conditions, classifying each proved output into one of six categories (Table 9). Inter-annotator agreement was substantial ($\kappa = 0.72$).

Four findings emerge:

Diverse hacking modes. GRPO training produces at least four distinct structural degeneracy patterns: *tautology injection* (adding $P \vee \neg P$ as premises), *vacuous quantification* ($\forall x(P(x) \rightarrow Q(x))$ where P is never instantiated), *premise redundancy* (repeating premises in different syntactic forms), and *trivial conclusion* (concluding with tautologies regardless of premises).

STA reward does not selectively suppress hacking. The distributions are statistically indistinguishable between exec-only and exec+STA (χ^2 test, $p = 0.68$). STA reward uniformly suppresses prove rate rather than targeting specific hacking modes.

Differential ISR sensitivity. ISR detects premise redundancy most reliably (90%) because removing redundant premises does not change the solver verdict. Tautology injection is well-detected (85%). However, trivial conclusion is poorly detected (30%) because conclusion triviality does not necessarily manifest as component removability—the conclusion may still be needed for the proof to proceed, even if it is trivially true.

Overall ISR diagnostic sensitivity. Across all hacking modes, ISR achieves $\sim$70% detection sensitivity (Table 10), validating its utility as a post-hoc diagnostic tool.

Category	Exec%	STA%	ISR Det.
Tautology injection	28	22	85%
Vacuous quantification	19	21	60%
Premise redundancy	14	15	90%
Trivial conclusion	8	7	30%
Correct structural	26	30	—
Other degenerate	5	5	variable

Table 9: Reward hacking taxonomy (200 samples per condition). Hacking pattern distributions are nearly identical between conditions (χ^2 test $p = 0.68$), confirming that STA reward does not selectively suppress any hacking mode. ISR Det. = percentage of instances detected by ISR.

Hacking Mode	n	Detected	Missed
Tautology injection	56	85%	15%
Vacuous quantification	38	60%	40%
Premise redundancy	28	90%	10%
Trivial conclusion	16	30%	70%

Table 10: ISR detection sensitivity by hacking mode. ISR is most effective at detecting premise redundancy and tautology injection, but struggles with trivial conclusions.

6 Discussion

Why does STA fail as training reward? The core issue is signal sparsity in the low-prove-rate regime. ISR is defined only for proved outputs; with prove rate $< 20\%$, roughly 80% of rollouts contribute zero ISR gradient. Among the remaining 20%, only $\sim$30% have non-trivial ISR values, yielding an effective per-step sample size of $K \times 0.20 \times 0.30 \approx 0.5$ informative samples. This extreme sparsity causes two pathologies: (1) high gradient variance that destabilizes policy optimization, and (2) reward avoidance where the model learns to generate unprovable-but-valid outputs to avoid ISR penalties.

When might STA-as-reward work? Our analysis suggests a clear prerequisite: base prove rate must be sufficiently high ($>$50%) for ISR to provide enough samples per step. This could be achieved with stronger base models, curriculum-based prove-rate bootstrapping, or easier formalization tasks. STA-as-reward is not inherently flawed—it is premature for the current capability level.

STA as diagnostic retains value. Even as the training reward fails, the ISR metric and hacking taxonomy provide actionable insights invisible to execution accuracy alone. The ISR/PR ratio tracks structural quality during training, revealing that the base model (ISR/PR = 0.53) has better structural properties than either GRPO-trained model—a finding with implications for when to prefer SFT over GRPO.

GRPO amplifies solver shortcuts. The SFT comparison reveals that GRPO's on-policy exploration inherently discovers and reinforces solver-exploitable patterns. SFT, constrained to imitate gold translations, achieves both higher accuracy and better structural quality at a fraction of the training cost (12 minutes vs. 78–84 minutes). This suggests that for formalization tasks with available gold data, GRPO may be strictly worse than SFT unless equipped with sufficiently informative structural rewards.

Limitations. Our study is limited to a single logic type (FOL+Prover9) due to the Prolog instantiation failure and SMT-LIB being deprioritized. The base model (Qwen2.5-1.5B) is relatively small; larger models may achieve higher base prove rates that make STA-as-reward viable. The hacking taxonomy, while systematic ($\kappa = 0.72$), is based on 200 samples and may not capture all degeneracy modes.

7 Conclusion

We introduced Solver Trace Ablation (STA), a framework grounded in mutation testing's RIP model for deriving structural rewards from solver execution semantics. Our principled negative result demonstrates that while STA effectively diagnoses reward hacking—detecting $\sim$70% of degenerate outputs across four identified hacking modes—it fails as a training reward in the current low-prove-rate regime, actively reducing prove rate from 0.24 to 0.11 without improving structural quality.

The key contributions are: (1) the 5-step STA derivation protocol and ISR metric as diagnostic tools for structural quality assessment; (2) a mechanistic explanation of why dense structural rewards fail under sparse primary signal (gradient variance amplification); and (3) a 200-sample hacking taxonomy providing the first fine-grained characterization of reward hacking in GRPO-trained formalizers.

Our results point to a clear research direction: STA-as-reward requires a prerequisite base prove rate that current small models on hard formalization tasks do not meet. Future work should investigate STA with stronger base models, curriculum-based bootstrapping, or hybrid SFT-then-GRPO pipelines that establish sufficient prove rate before introducing structural rewards.

References

Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger. Defining and characterizing reward hacking. In *NeurIPS*, 2022.

L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. In *ICML*, 2023.

Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.